QC SKILL · RECONSTRUCTION SPECIFICATION

qc-coherence

Unbounded codebase-wide verification. Finds the drift two clean missions still leave behind.

Use before a release, after 5+ feature sessions, when a new subsystem landed, or when per-function checks come back clean but the tree feels inconsistent.
Original: ~/.claude/skills/qc-coherence/ · --scope=module absorbs the former qc-structural skill
August 2026 · Port this document; do not weaken the laws.

Contents

1. Purpose

Hardening guarantees quality inside individual functions. Coherence guarantees consistency *across the entire codebase*. Two clean missions can still leave naming drift, protocol gaps, duplicated logic, stubs, dangling references, and layer violations. This skill has no mission scope. It runs less often than hardening — typically before releases or after major feature cycles.

Skill	Scope	Question
qc-hardening	Per-function	Does each function work correctly?
qc-coherence --scope=module	Per-module	Do modules compose correctly?
qc-coherence (default)	Full codebase	Is the codebase internally consistent?
qc-all	All of the above	Is the codebase release-ready?

Triggers: coherence audit, coherence check, cross-cutting audit, codebase coherence. **Do not use** during active feature work, or when a targeted hardening pass is the better tool.

2. Pre-flight and scopes

1. Read project conventions (CLAUDE.md / AGENTS.md, pyproject.toml / package.json).
2. Load prior state: .structural-integrity.md, .hardening-profile.md, last coherence ledger.
3. Build module inventory with LOC.
4. Check git history since the last coherence audit.

5. Determine scope: default --scope=codebase (four sub-audits) or --scope=module (MS-1-MS-6). Focus paths may be caller-specified.

Default execution order (codebase): Structural (deterministic, no LLM) then Conformance, Semantic, Architectural. Each later sub-audit consumes the symbol table and import graph from Structural. Module-scope can run instead of, or in addition to, the four sub-audits.

3. Four codebase sub-audits

S — Structural (deterministic, always inline)

Fast, no LLM. Produces the clean symbol table and import graph later sub-audits consume. Tools: ruff, mypy --strict / tsc --strict, vulture if available, AST import graph.

Check	Finds	Severity notes
S1 Stub detection	TODO/FIXME/HACK/XXX; raise NotImplementedError outside abstracts; pass- or ...-only non-protocol methods	P1 production path; P0 if reachable unimplemented method; P3 test-only. Do not flag abc.abstractmethod, Protocol stubs, TYPE_CHECKING, intentional empty fixtures.
S2 Dead code / dangling refs	Orphan __all__ exports, orphan public defs, broken imports, shadowed imports	Broken import = P0 (import-time crash). Else P2. Entry points and __init__ re-exports are consumers, not dead.
S3 Duplication	Exact clones always; parametric clones if 3+; semantic clones if matching tests. Threshold: 6+ lines x2 or 3+ lines x3	Shared util exists but unused = P1. Else P2. Exempt test isolation and framework boilerplate.
S4 Type coverage	Missing public annotations, Any, type: ignore without comment, checker errors	Checker error = P0; bare type: ignore = P1; else P2. Floor 95% on production; ratchet only up.
S5 Unused dependencies	Manifest packages with zero imports (map Pillow→PIL etc.)	P3. Exempt build/dev tools, plugins, transitives.

Structural-candidate tiering for the workflow: only route AMBIGUOUS hits (S1 stubs, S3 duplication, S2 orphan exports/defs) through adversarial verify — those can be deliberate seams. Mechanically certain hits (broken imports, mypy errors, unused deps) are findings directly; do not spend two agents proving a broken import.

C — Conformance

Universal protocol/implementation matrix. Discover interfaces (ABC, Protocol, __subclasshook__, docstring-declared contracts) and every concrete implementation (explicit, register(), structural duck-typing). Build a cell per method: YES, MISSING, STUB, SIGNATURE-MISMATCH, PARTIAL.

- MISSING: P0 if that implementation is deployed, else P1.
- STUB or SIGNATURE-MISMATCH: P1. PARTIAL (missing contracted error handling): P2.
- Every interface with 2+ implementations must have parameterized conformance tests covering ALL of them. Missing test or missing impl in the parameter list = P1. Generate a test skeleton as the remediation artifact.
- For interfaces that pass today, emit a drift-guard test that fails when the interface gains methods the impl does not have.

M — Semantic (LLM-assisted)

Check	Question	Sev
M1 Naming	Same domain concept, different names (run_id vs execution_id; txn vs transaction; DB column vs Python attr vs API field). Cluster, pick canonical (most frequent or protocol-defined), flag variants.	P2
M2 Error taxonomy	Equivalent conditions raising different exception types, or missing handling on external calls. Cluster raise-sites by condition semantics.	P1 missing; P2 type inconsistency
M3 Serialization round-trip	deserialize(serialize(x)) == x. Key mismatch / asymmetry = P1; missing round-trip test = P2. Watch optional fields, enums, datetimes, JSON-blob columns.	P1 / P2
M4 Behavioral contract	Same interface method, different behavior across implementations (None/empty, invalid input, return types, side effects). Bug = P1; underspecified interface = P2.	P1 / P2
M5 Magic values	Hardcoded literals with domain meaning, used in 2+ files, not centralized. Status strings used as ad-hoc enums.	P2

A — Architectural

Check	Question	Sev
A1 Dependency direction	Lower layers must not import higher. Infer layers from docs/ADRs or import graph. Upward or circular = violation. Runtime = P1; TYPE_CHECKING-only = P2. Missing layer docs = P2. Emit import-linter rules when available.	P1 / P2
A2 Layer boundary	Public API vs internals. Flag reaching across packages into _privates or bypassing __init__. Infrastructure imports (sqlalchemy, requests) inside domain/core = P1.	P1 / P2
A3 Module responsibility	Modules >500 LOC: count public symbols and responsibility domains. 2+ domains = mixed (P2). >1200 LOC with 5+ tables = P1. Propose split; do not execute it in this check.	P1 / P2
A4 Documented invariants as tests	Scan comments/docs/ADRs for must/never/invariant/ordering. Testable runtime invariant without a test = P1; static = P2.	P1 / P2
A5 Cross-cutting consistency	Logging, auth, retry, transactions, caching: shared util used everywhere, same parameters. Auth/transaction inconsistency = P1; else P2.	P1 / P2

When the audit finds a pattern worth enforcing permanently, emit architectural rules as remediation artifacts (not auto-committed): import-linter contracts, pytest structural fixtures, pre-commit / linter config. Each rule cites the finding and states the invariant.

4. Module scope (--scope=module)

These six checks (formerly qc-structural) ask whether modules and protocol implementations still compose. Finding IDs use MS-N.N. They appear in the main FINDINGS list and use the same P0-P3 scale.

Check	Question / thresholds	Fix posture
MS-1 Protocol conformance parity	Same assertions against every backend. P0: Protocol method has no conformance test. P1: test covers a subset.	Write missing tests. Do not change production code in this check.
MS-2 Responsibility density	>800 LOC + 3+ unrelated responsibilities = P1. >1200 LOC + 5+ tables behind one lock = P0.	Propose a split with boundaries. Do not execute it.
MS-3 Coordination surface	3+ sync primitives interacting = P1. Path that produced 2+ hardening findings = P0. Inconsistent lock order = P0 (deadlock).	Document state machine and lock order. Simplify only if behavior-preserving.

Check	Question / thresholds	Fix posture
MS-4 Backend parity	P0: Protocol method on one backend not the other (works in dev, breaks in prod). P1: duplicated shared logic. Also check SQL dialect assumptions (LIMIT/OFFSET vs OFFSET/FETCH, ON CONFLICT vs MERGE).	Add missing methods; extract shared helpers; add conformance tests.
MS-5 Cross-session consistency	Grep new code for patterns the hardening profile already flagged. Reintroduced known-bad pattern = P0.	Fix reintroduced bugs immediately. Convention drift goes to pass-debt.
MS-6 Test architecture	P0: test file >1500 LOC or production module with zero tests. P1: fixture duplication across 3+ files or test file >1000 LOC.	Extract fixtures; split by domain; add missing test files.

Accepted debt staleness: every entry in .structural-integrity.md needs Last reviewed: YYYY-MM-DD. Unreviewed 30+ days = STALE (P1). Unreviewed 60+ days is promoted from accepted debt to P1 — it must be re-accepted or resolved.

5. Discovery+Verify workflow

Preferred for codebase scope (highest yield >5K LOC or after several changes). Structural stays inline and tool-driven. Conformance, Semantic, and Architectural run as workflows/coherence-discovery-verify.js. Above ~50K LOC pass moduleGroups so each lens fans out per group and does not skim.

```
Workflow({
  scriptPath: "<skill>/workflows/coherence-discovery-verify.js",
  args: {
    repoPath, scope, conventions,
    structuralContext,          // symbol table / import graph from S
    structuralCandidates,       // AMBIGUOUS S1/S2/S3 hits only
    moduleGroups,               // optional; required >~50K LOC
    moduleInventory, priorState, offLimits
  }
})
```

Phases: Audit (one agent per lens, optionally per group) → Verify (**two perspective-diverse votes**: an intentionality hunter looking for the seam doc / layer rationale that makes the divergence deliberate, plus a mechanics validator confirming cited locations exist and severity is right) → Synthesize (dedup, rank, coverage vs inventory). Tie-handling is strict: recommendation is **fix** only when BOTH verdicts say real; a split or any uncertain → **defer**, never silently dropped. **drop** is verified-intentional or overstated (keep intentional_evidence). The workflow never edits or commits. Convergence (which site is canonical, whether to split a module) stays with the parent. Use coverage to confirm every lens returned a CLEAN list.

If the Workflow tool is unavailable, fall back to per-module-group subagents with the same “report only, do not fix” rule, then coordinator dedup and apply.

6. Severity, verdicts, ledger

Sev	Definition
P0	Actively broken: runtime failure, data corruption, security hole.
P1	Will break under realistic conditions: missing error handling, unimplemented protocol method.
P2	Increases maintenance cost: naming drift, duplication, undocumented invariant.
P3	Suboptimal but functional: style inconsistency, minor dead code.

Prose report vs JSON rollup — map them; do not invent a fourth enum

Prose verdict	JSON verdict	Condition
COHERENT	READY	Zero findings (0 P0, 0 P1 in the prose banner; JSON READY is zero findings).
DRIFTING	READY_WITH_DEBT	Working today but accumulating risk. JSON: only P2/P3, all fixed or deferred_because set.
ACTION REQUIRED	NOT_READY	Fix before continuing development. JSON: any P0/P1 with fixed:false and no deferred_because.

Write `.qc-findings/qc-coherence.json` to the shared qc-finding schema (schema_version 1.0, skill qc-coherence, run_id, git_sha, findings, verdict). When findings come from the workflow, translate fields explicitly:

Workflow ledger	qc-finding.json
check (+ sequence)	id (e.g. M1.2)
locations[0]	file, line (remaining locations go in what)
title	what (prefix; append the location list)
why_incoherent	why
remediation_hint (fallback proposed_fix)	fix
recommendation: fix, then applied	fixed: true
recommendation: defer	fixed: false + deferred_because
recommendation: drop	omit from the file entirely

7. Dedup, persistence, remediation

- Same location flagged by multiple sub-audits: keep highest severity, merge evidence, cite all source sub-audits.
- Overlap with an accepted-debt entry in .structural-integrity.md: note it; do not re-flag unless new evidence changes severity.
- Overwrite .qc-coherence-report.md each run (current state, not history).
- **Diff mode:** copy previous .qc-findings/qc-coherence.json to .qc-coherence-baseline.json and pass it as priorState so lenses suppress already-recorded findings unless they regressed.

Verdict	Supervisor action
COHERENT	No dispatch. Log and move on.
DRIFTING	Dispatch autonomous supervisor with P1 + P2 findings.
ACTION REQUIRED	Dispatch with ALL findings. Block further feature work.

Dispatch target: .github/agents/autonomous-supervisor.agent.md. Payload format lives in qc-all/references/supervisor-dispatch.md. Convergence (canonical name, module split) is a design call the fan-out cannot make — that is why remediation stays model-driven.

8. Files to vendor and minimum viable port

Path	Role
SKILL.md	Trigger, scopes, sub-audits, verdicts, ledger mapping.
references/structural-audit.md	S1-S5.
references/conformance-audit.md	C1 matrix + drift guard.
references/semantic-audit.md	M1-M5.
references/architectural-audit.md	A1-A5 + rule generation.
references/module-scope-checks.md	MS-1-MS-6 + debt staleness.
references/structural-checks-migration.md	qc-structural → --scope=module.
workflows/coherence-discovery-verify.js	Optional. LLM-assisted half.
qc-finding.schema.json + verify_ledger.py	From qc-all. Copy if standalone.

Do not invent on a port

- Do not treat a deliberate seam (documented layer exception, per-backend dialect) as drift without the intentionality check.
- Do not drop findings on a split verify vote — defer them.
- Do not execute module splits inside MS-2; propose boundaries.
- Do not skip Structural just because a workflow exists — the workflow needs its symbol table.
- Do not emit SKIPPED from a run that actually executed.